# LAB 05 — SUBMISSION REPORT

SE3082-Parallel Computing

**IT24103440**

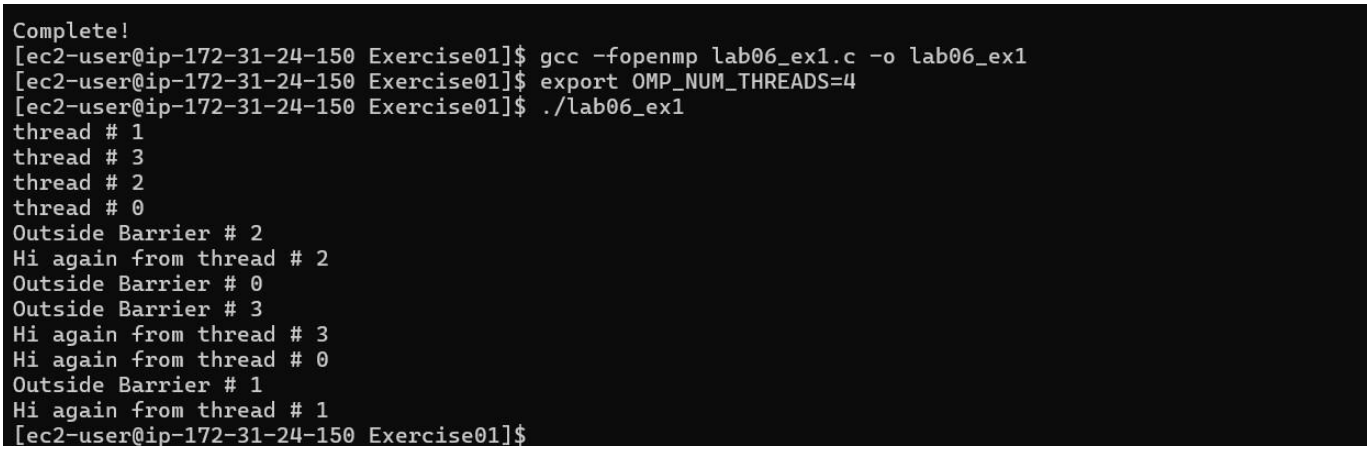
**Environment visible in evidence:** Linux / AWS EC2 terminal, GCC with -fopenmp.

# Exercise 1: OpenMP Barrier

**Objective and explanation.** The barrier makes every thread wait until all participating threads have arrived. The screenshot shows the threads reaching the barrier before displaying output after it. Output ordering between threads is nondeterministic.

**Answer:** pragma omp barrier forces all threads to stop at that point until every thread reaches the barrier. After all threads arrive, they continue execution together.

## Screenshot



# Exercise 2: OpenMP Single

**Objective and explanation.** Only one thread executes the single region; other threads skip the block. An implicit barrier occurs at the end of single unless nowait is specified.

**Answer:** #pragma omp single allows only one thread to execute the block. The remaining threads skip that block and continue.

## Screenshot

```
[ec2-user@ip-172-31-24-150 Exercise01]$ cd ../ && mkdir -p Exercise02 && cd Exercise02
[ec2-user@ip-172-31-24-150 Exercise02]$ nano lab06_ex2.c
[ec2-user@ip-172-31-24-150 Exercise02]$ gcc -fopenmp lab06_ex2.c -o lab06_ex2
[ec2-user@ip-172-31-24-150 Exercise02]$ ./lab06_ex2
thread # 1
Inside Single # 1
Exiting Single
thread # 3
thread # 0
thread # 2
Hi again from thread # 0
Hi again from thread # 2
Hi again from thread # 1
Hi again from thread # 3
[ec2-user@ip-172-31-24-150 Exercise02]$
```

# Exercise 3: OpenMP Master

**Objective and explanation.** The master construct is executed only by the primary thread (thread 0). Unlike single, master has no implied barrier.

**Answer:** #pragma omp master allows only the master thread (thread 0) to execute the block.

## Screenshot

```
[ec2-user@ip-172-31-24-150 Exercise02]$ cd ../ && mkdir -p Exercise03 && cd Exercise03
[ec2-user@ip-172-31-24-150 Exercise03]$ nano lab06_ex3.c
[ec2-user@ip-172-31-24-150 Exercise03]$ gcc -fopenmp lab06_ex3.c -o lab06_ex3
[ec2-user@ip-172-31-24-150 Exercise03]$ ./lab06_ex3
thread # 3
Hi again from thread # 3
thread # 1
Hi again from thread # 1
thread # 0
Inside Master # 0
thread # 2
Exiting Master
Hi again from thread # 2
Hi again from thread # 0
[ec2-user@ip-172-31-24-150 Exercise03]$
```

# Exercise 4: Nested Loops and collapse(2)

**Objective and explanation.** Without collapse, parallel for divides five outer iterations (r=1..5) among threads, and each assigned thread runs its ten inner iterations. With collapse(2), the 50 logical iterations become a single worksharing iteration space. The archive shows a screenshot for the original nested-loop run, not a separate collapse(2) run.

**Answer:** Without collapse: #pragma omp parallel for Only the outer loop iterations are divided among threads.

Example:
Thread 0 → r=1
Thread 1 → r=2
Thread 2 → r=3

With: #pragma omp parallel for collapse(2)
both loops become one large loop:

5 × 10 = 50 iterations and
threads share all 50 tasks.

## Screenshot

```
[ec2-user@ip-172-31-24-150 Exercise03]$ cd ../ && mkdir -p Exercise04 && cd Exercise04
[ec2-user@ip-172-31-24-150 Exercise04]$ nano lab06_ex4.c
[ec2-user@ip-172-31-24-150 Exercise04]$ gcc -fopenmp lab06_ex4.c -o lab06_ex4
[ec2-user@ip-172-31-24-150 Exercise04]$ export OMP_NUM_THREADS=4
[ec2-user@ip-172-31-24-150 Exercise04]$ ./lab06_ex4
r = 4, k = 9, thread # = 3
r = 4, k = 10, thread # = 3
r = 5, k = 1, thread # = 3
r = 5, k = 2, thread # = 3
r = 5, k = 3, thread # = 3
r = 5, k = 4, thread # = 3
r = 5, k = 5, thread # = 3
r = 5, k = 6, thread # = 3
r = 5, k = 7, thread # = 3
r = 5, k = 8, thread # = 3
r = 2, k = 4, thread # = 1
r = 2, k = 5, thread # = 1
r = 2, k = 6, thread # = 1
r = 2, k = 7, thread # = 1
r = 2, k = 8, thread # = 1
r = 2, k = 9, thread # = 1
r = 2, k = 10, thread # = 1
r = 3, k = 1, thread # = 1
r = 3, k = 2, thread # = 1
r = 3, k = 3, thread # = 1
r = 3, k = 4, thread # = 1
r = 3, k = 5, thread # = 1
r = 3, k = 6, thread # = 1
r = 5, k = 9, thread # = 3
r = 3, k = 7, thread # = 2
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
```

# Exercise 5: Task-parallel Fibonacci

**Objective and explanation.** Two OpenMP tasks compute fib(n-1) and fib(n-2), and taskwait ensures both have finished before their results are combined. A single thread begins the recursive computation inside a parallel region. Fine-grained tasks can create substantial overhead.

## Source code

```
#include <stdio.h>
#include <omp.h>

int fib(int n) {     int i, j;     if (n < 2)
return n;     else {          #pragma omp task
shared(i) firstprivate(n)          i = fib(n - 1);

        #pragma omp task shared(j) firstprivate(n)
j = fib(n - 2);

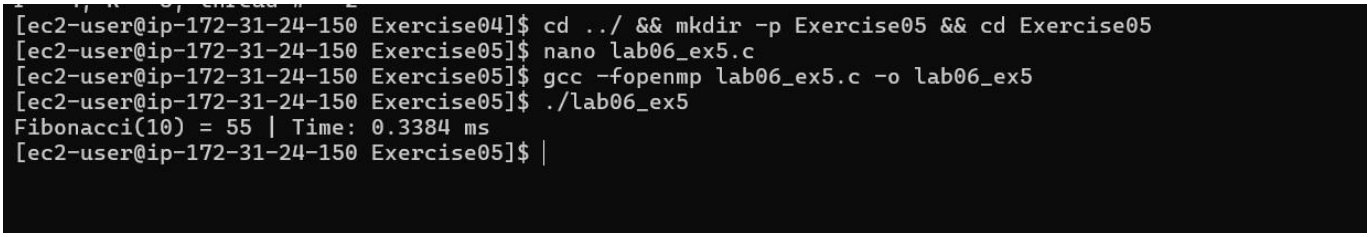
        #pragma omp taskwait
return i + j;     } }

int main() {     int n = 10, result;
double tstart = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single
{              result =
fib(n);          }     }

    double tstop = omp_get_wtime();     printf("Fibonacci(%d) = %d | Time: %.4f ms\n", n,
result, (tstop - tstart) * 1000.0);     return 0; }
```

## Screenshot



# Exercise 6: Strip-mined Array Multiplication

**Objective and explanation.** The one-million-element arrays are divided into 256-element strips and those strips are distributed by parallel for. Each strip computes C[i]=A[i]*B[i]; because the arrays are initialized with 1.5 and 2.0, all valid outputs should equal 3.0.

## Source code from archive

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000000 #define STRIP_SIZE 256  // Aligned with typical
SIMD / block widths

int main() {     double *A = (double*)malloc(N *
sizeof(double));     double *B = (double*)malloc(N *
sizeof(double));     double *C = (double*)malloc(N *
sizeof(double));

    for (int i = 0; i < N; i++) {
A[i] = 1.5;
        B[i] = 2.0;
}

    double tstart = omp_get_wtime();

    // Strip mining technique     #pragma omp parallel for     for
(int ii = 0; ii < N; ii += STRIP_SIZE) {         int limit = (ii +
STRIP_SIZE < N) ? (ii + STRIP_SIZE) : N;         for (int i = ii; i
< limit; i++) {             C[i] = A[i] * B[i];
        }
}

    double tstop = omp_get_wtime();     printf("Array multiplication completed. Time: %.4f
ms\n", (tstop - tstart) * 1000.0);

    free(A); free(B); free(C);
return 0; }
```

## Execution screenshot



# Observations and conclusion

The screenshots indicate that six exercises were compiled and run in a Linux EC2 terminal. The first four illustrate OpenMP synchronization and worksharing; Exercises 5 and 6 provide C implementations for task parallelism and strip mining.